

Compte Rendu TD9 : Protéger une application web en appliquant un codage sécurisé

Sommaire

Introduction.....	1
Étape 1.....	2
1.....	2
2.....	2
3.....	2
4 – Démarrage des machines virtuelles.....	3
Étape 2.....	3
5 – Réalisation de l'injection SQL (niveau de sécurité 0).....	3
Étape 3.....	3
6 – Contre-mesure : passage au niveau de sécurité 5.....	3
7 – Comparaison du code source sécurisé vs non sécurisé.....	4
Conclusion.....	4

Introduction

Dans ce TP, nous avons appris à protéger une application web contre les attaques informatiques.

Pour cela, nous avons utilisé des machines virtuelles et le site Mutillidae afin de tester une injection SQL. Nous avons comparé une application sans sécurité et une application protégée pour voir la différence.

Ce travail m'a permis de mieux comprendre l'importance du codage sécurisé dans le développement web.

Étape 1

1.

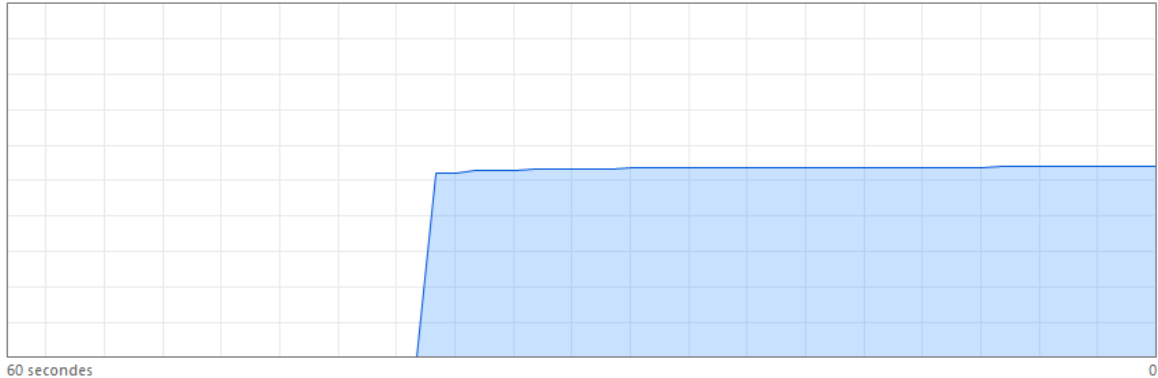
J'ai commencé par vérifier que l'ordinateur avait bien 6go de ram

Mémoire

Utilisation de la mémoire

16,0 Go

13,9 Go

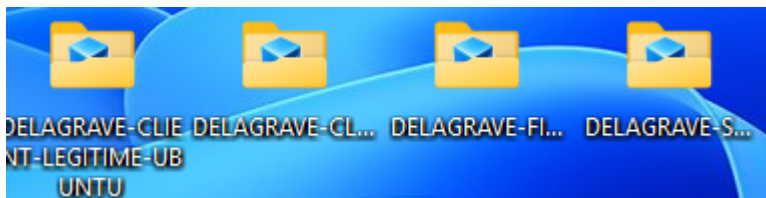


le pc dispose de 16go de ram c'est donc bon !

Je n'ai pas eu besoin de télécharger et installer VirtualBox car je l'ai déjà sur le pc et je m'en sert .

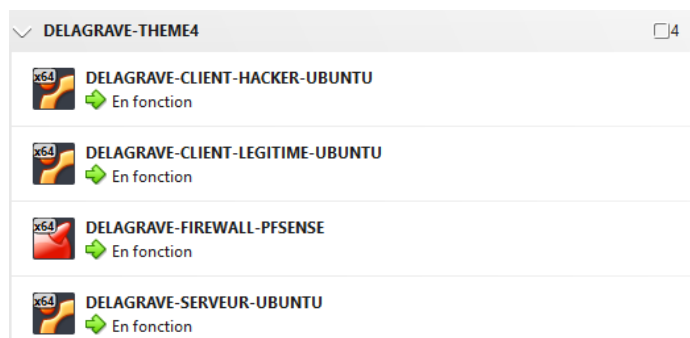
2.

J'ai télécharger le fichier DELAGRAVE-LAB-THEME4.OVA



3.

J'ai ouvert les VM sur VirtualBox



4 – Démarrage des machines virtuelles

J'ai démarré la machine DELAGRAVE-CLIENT-LEGITIME-UBUNTU en premier. Depuis son bureau, j'ai lancé la vidéo LAB-THEME4-CONFIG.mkv et j'ai suivi les instructions pour configurer les cartes réseau des quatre machines. Une fois la configuration terminée, j'ai démarré toutes les machines virtuelles dans VirtualBox.

Étape 2

5 – Réalisation de l'injection SQL (niveau de sécurité 0)

Depuis la machine DELAGRAVE-CLIENT-HACKER-UBUNTU, j'ai ouvert le fichier LAB-THEME4-CHAP6-DEFI.txt qui se trouvait sur le bureau. Il contenait les instructions pour réaliser une injection SQL sur la page user-info.php du site Mutillidae, avec le niveau de sécurité à 0.

J'ai suivi les étapes indiquées et j'ai constaté que l'injection SQL a fonctionné : la liste complète des membres du site s'est affichée. Cela signifie qu'avec le niveau 0, il n'y a aucune protection, et n'importe qui peut accéder à des données confidentielles auxquelles il ne devrait pas avoir accès.

Étape 3

6 – Contre-mesure : passage au niveau de sécurité 5

J'ai ensuite changé le niveau de sécurité de Mutillidae à 5 en cliquant sur le bouton "Toggle Security". J'ai rejoué exactement la même injection SQL que précédemment.

Cette fois, l'attaque n'a pas fonctionné. Le site a bloqué la requête malveillante et n'a affiché aucune donnée sensible. Le codage sécurisé a donc bien joué son rôle de protection.

7 – Comparaison du code source sécurisé vs non sécurisé

En comparant le code source de la page user-info.php entre le niveau 0 et le niveau 5, j'ai remarqué plusieurs différences.

Au niveau 5, le code effectue deux vérifications supplémentaires avant d'exécuter la requête SQL :

- Il vérifie la longueur des données saisies pour éviter qu'un texte trop long (contenant du code malveillant) soit accepté.
- Il filtre le contenu saisi grâce à une liste noire de caractères interdits (comme les guillemets, tirets, etc.) souvent utilisés dans les injections SQL.

J'en déduis que les bonnes pratiques de codage sécurisé consistent à ne jamais faire confiance aux données saisies par l'utilisateur : il faut toujours les contrôler, les limiter en taille et filtrer les caractères dangereux avant de les utiliser dans une requête SQL.

Conclusion

Ce TP m'a permis de comprendre comment une injection SQL peut fonctionner sur une application mal protégée.

J'ai aussi vu que les protections mises en place permettent de bloquer ce type d'attaque et de sécuriser les données.

Grâce à cette activité, j'ai compris qu'il est important de toujours vérifier les données saisies par les utilisateurs pour éviter les failles de sécurité.